Cloud-based Exploration on the Hugging Face platform for Receipt Extraction Task

Maritza Amalia Dwiputri - 203012610017

About Hugging Face

Hugging Face is an **open-source platform** where developers share and collaborate on machine learning models, datasets, and AI tools — often called the "GitHub of Machine Learning." (*https://blog.udemy.com/what-is-hugging-face-guide/*)



Hugging Face

What does Hugging Face do?

- It hosts over 2M models, 1.5M datasets, and 1.5M AI apps (Spaces), all open and publicly available.
- It provides Machine Learning Libraries; libraries to fetch data, load pretrained backbones, preprocess, and run forward passes.
- It provides tools for building interfaces, running agents, and managing specialized workflows.

Utilize Hugging Face

Hub	Hugging Face Hub hosts the open-source model (Qwen2.5-VL-3B) and the dataset (CORD-v2).
Libraries	Transformers ⇒ model-definition framework Datasets ⇒ accessing dataset Parameter-Efficient Fine-Tuning (PEFT) ⇒ enable efficient adaptation of large pretrained models Accelerate ⇒ handles where and how PyTorch code runs TRL ⇒ supervised fine-tuning loop
Others	bitsandbytes ⇒ (non hugging face) storing numbers in fewer bits OpenCV ⇒ (non hugging face) pre process image gradio ⇒ build web interface

Cloud Environment



NVIDIA T4 GPU with 16 GB VRAM inside Google Colab Pro.
The T4 has no native bf16, so computation uses fp16

Exploration: Dataset

I use **naver-clova-ix/cord-v2**, receipt extractor dataset, that has 800 train, 100 validation, and 100 test data.




```
{
  "menu":
    {
      "nm": "DONAT GULA",
      "unitprice": "@11000",
      "cnt": "1X", "price": "11,000"
    },
  "sub_total":
    {
      "subtotal_price": "11,000"
    },
  "total":
    {
      "total_price": "11,000",
      "cashprice": "11,000",
      "menuqty_cnt": "1.00xITEMs"
    }
}
```


Dataset Example

Load data using load_dataset from datasets.

```
from datasets import load_dataset

dataset = load_dataset('naver-clova-ix/cord-v2')
```

It directly download dataset from Hugging Face Hub to the project.

Exploration: Model

I use **Qwen2.5-VL-3B**, a SOTA Visual Language Model, that has 3 Billion parameters.




```
vision encoder (incl. projector): Qwen2_5_VisionTransformerPretrainedModel 335,351,808
projector (visual.merger)       : Qwen2_5_VLPatchMerger 18,358,528
language model                  : Qwen2_5_VLTextModel 1,698,672,640
lm_head                         : Linear 311,164,928 (may be tied to the embedding table)

Qwen2_5_VLPatchMerger(
  (ln_q): Qwen2_5_VLRMSNorm((1280,), eps=1e-06)
  (mlp): Sequential(
    (0): Linear4bit(in_features=5120, out_features=5120, bias=True)
    (1): GELU(approximate='none')
    (2): Linear4bit(in_features=5120, out_features=2048, bias=True)
  )
)
```

Load data using Qwen2_5_VLForConditionalGeneration, BitsAndBytesConfig, AutoProcessor from transformers.

```python
from transformers import Qwen2_5_VLForConditionalGeneration, AutoProcessor,
BitsAndBytesConfig

MODEL_NAME = 'Qwen/Qwen2.5-VL-3B-Instruct'

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True, bnb_4bit_quant_type='nf4',
    bnb_4bit_use_double_quant=True,
    bnb_4bit_compute_dtype=torch.float16,
    llm_int8_skip_modules=['visual', 'merger', 'lm_head'],
)

hf_model = Qwen2_5_VLForConditionalGeneration.from_pretrained(
    MODEL_NAME, quantization_config=bnb_config, torch_dtype=torch.float16,
    device_map={'': 0}, attn_implementation='sdpa')

processor = AutoProcessor.from_pretrained(MODEL_NAME)
```

Exploration: OpenCV Pre Processing

I use OpenCV (non Hugging Face) to pre process image which is shrinking a receipt photo to a size the model can handle, without distorting.

```
import cv2
import numpy as np
from PIL import Image

def preprocess(pil_img, max_side=MAX_SIDE):
    img = cv2.cvtColor(np.array(pil_img.convert('RGB')), cv2.COLOR_RGB2BGR)
    h, w = img.shape[:2]
    scale = max_side / max(h, w)
    if scale < 1:
        img = cv2.resize(img, (int(w * scale), int(h * scale)), interpolation=cv2.INTER_AREA)
    return Image.fromarray(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
```

I resize the photo for two reasons: the model can only fit images up to a certain size in GPU memory, and the text has to stay readable after shrinking.

Exploration: OpenCV Pre Processing (2)

Original Image
Size: (960, 1280)



Preprocessed Image
Size: (600, 800)



Exploration: Custom Assembly VLM Model (1)

Although this project still use the same Qwen2.5-VL-3B architecture, I decide to assembly it manually so in the future I can change elements inside the architecture.

```python
class AssembledVLM(torch.nn.Module):
    def __init__(self, visual, text_model, lm_head, embed_tokens, rope_fn, image_token_id):
        super().__init__()
        self.visual, self.text_model, self.lm_head = visual, text_model, lm_head
        self.embed_tokens, self.rope_fn, self.image_token_id = embed_tokens, rope_fn, image_token_id

    def _image_embeds(self, pixel_values, image_grid_thw):
        dtype = next(self.visual.parameters()).dtype
        out = self.visual(pixel_values.to(dtype), grid_thw=image_grid_thw)
        if not isinstance(out, torch.Tensor):
            pooled = getattr(out, 'pooler_output', None)
            out = pooled if pooled is not None else out[0]
        return out

    def _inputs_embeds(self, input_ids, pixel_values, image_grid_thw):
        image_embeds = self._image_embeds(pixel_values, image_grid_thw)
        emb = self.embed_tokens(input_ids)
        mask = input_ids == self.image_token_id
        if int(mask.sum()) != image_embeds.shape[0]:
            raise ValueError(f'{int(mask.sum())} placeholder tokens but {image_embeds.shape[0]} image vectors')
        return emb.masked_scatter(mask.unsqueeze(-1).expand_as(emb), image_embeds.to(emb.dtype))

    def _positions(self, input_ids, attention_mask, image_grid_thw, mm_token_type_ids=None):
        kwargs = dict(input_ids=input_ids, image_grid_thw=image_grid_thw, attention_mask=attention_mask)
        if 'mm_token_type_ids' in inspect.signature(self.rope_fn).parameters:
            if mm_token_type_ids is None:
                mm_token_type_ids = (input_ids == self.image_token_id).long()
            kwargs['mm_token_type_ids'] = mm_token_type_ids
        return self.rope_fn(**kwargs)

    ...
```

```python
class AssembledVLM(torch.nn.Module):
    ...

    def _hidden(self, input_ids, attention_mask, pixel_values, image_grid_thw, mm_token_type_ids=None):
        emb = self._inputs_embeds(input_ids, pixel_values, image_grid_thw)
        position_ids, _ = self._positions(input_ids, attention_mask, image_grid_thw, mm_token_type_ids)
        out = self.text_model(input_ids=None, inputs_embeds=emb, position_ids=position_ids,
                              attention_mask=attention_mask, use_cache=False)
        return out.last_hidden_state if hasattr(out, 'last_hidden_state') else out[0]

    def logits(self, input_ids, attention_mask, pixel_values, image_grid_thw, mm_token_type_ids=None, last_n=None):
        h = self._hidden(input_ids, attention_mask, pixel_values, image_grid_thw, mm_token_type_ids)
        if last_n: h = h[:, -last_n:]
        return self.lm_head(h)

    def forward(self, input_ids, attention_mask, pixel_values, image_grid_thw, labels, mm_token_type_ids=None):
        h = self._hidden(input_ids, attention_mask, pixel_values, image_grid_thw, mm_token_type_ids)
        shift_h, shift_labels = h[:, :-1], labels[:, 1:]
        sel = shift_labels != -100
        return F.cross_entropy(self.lm_head(shift_h[sel]).float(), shift_labels[sel])

    def _pick(self, h, seen_ids, repetition_penalty=1.0):
        """Greedy choice of the next token. With repetition_penalty != 1 it copies the library's rule:
        every token already in the sequence (prompt included) has its logit divided by the penalty
        (multiplied if the logit is negative)."""
        logits = self.lm_head(h[:, -1:])[:, 0]
        if repetition_penalty != 1.0:
            score = logits.gather(1, seen_ids)
            score = torch.where(score < 0, score * repetition_penalty, score / repetition_penalty)
            logits = logits.scatter(1, seen_ids, score)
        return logits.argmax(-1, keepdim=True)

    ...
```

Exploration: Custom Assembly VLM Model (2)

Although this project still use the same Qwen2.5-VL-3B architecture, I decide to assembly it manually so in the future I can change elements inside the architecture.

```python
class AssembledVLM(torch.nn.Module):
    ...

    @torch.no_grad()
    def generate(self, input_ids, attention_mask, pixel_values, image_grid_thw, mm_token_type_ids=None,
                 max_new_tokens=1024, eos_token_ids=(), repetition_penalty=1.0):
        """Greedy decoding with a KV cache, written by hand.
        Prefill: run the whole prompt (image + text) once and keep the keys/values.
        Decode : feed ONE new token per step; its M-RoPE position is (index in sequence + rope_deltas),
                 the same offset for the 3 axes, because new tokens are plain text."""
        from transformers import DynamicCache
        emb = self._inputs_embeds(input_ids, pixel_values, image_grid_thw)
        position_ids, rope_deltas = self._positions(input_ids, attention_mask, image_grid_thw, mm_token_type_ids)
        cache = DynamicCache()
        out = self.text_model(input_ids=None, inputs_embeds=emb, position_ids=position_ids,
                              attention_mask=attention_mask, past_key_values=cache, use_cache=True)
        h = out.last_hidden_state if hasattr(out, 'last_hidden_state') else out[0]
        next_tok = self._pick(h, input_ids, repetition_penalty)
        generated, seq_len = [next_tok], input_ids.shape[1]
        for _ in range(max_new_tokens - 1):
            if int(next_tok) in eos_token_ids:
                break
            attention_mask = torch.cat([attention_mask, attention_mask.new_ones(1, 1)], dim=1)
            pos = (torch.full((1, 1), seq_len, device=input_ids.device) + rope_deltas).long()
            out = self.text_model(input_ids=None, inputs_embeds=self.embed_tokens(next_tok),
                                  position_ids=pos.unsqueeze(0).expand(3, -1, -1),
                                  attention_mask=attention_mask, past_key_values=cache, use_cache=True)
            h = out.last_hidden_state if hasattr(out, 'last_hidden_state') else out[0]
            next_tok = self._pick(h, torch.cat([input_ids] + generated, dim=1), repetition_penalty)
            generated.append(next_tok)
            seq_len += 1
        return torch.cat(generated, dim=1)
```

Exploration: Run baseline model

Since the task is about text generation, it uses prompt.

```
schema_dict = {
    'menu': {
        'nm': 'name of the menu', 'num': 'identification number of menu',
        'unitprice': 'unit price of menu', 'cnt': 'quantity of menu',
        'discountprice': 'discounted price of menu', 'price': 'total price of menu',
        'itemsubtotal': 'price of each menu after discount applied',
        'vatyn': 'whether the price includes tax or not', 'etc': 'others',
        'sub': {'nm': 'name of submenu', 'unitprice': 'unit price of submenu',
                'cnt': 'quantity of submenu', 'price': 'total price of submenu', 'etc': 'others'}
    },
    'sub_total': {
        'price': 'subtotal price', 'discount_price': 'discounted price in total',
        'service_price': 'service charge', 'othersvc_price': 'added charge other than service charge',
        'tax_price': 'tax amount', 'etc': 'others'
    },
    'total': {
        'total_price': 'total price', 'etc': 'others', 'cashprice': 'amount of price paid in cash',
        'changeprice': 'amount of change in cash',
        'creditcardprice': 'amount of price paid in credit/debit card',
        'emoneyprice': 'amount of price paid in emoney, point',
        'menutype_cnt': 'total count of type of menu', 'menuqty_cnt': 'total count of quantity'
    }
}

instruction = (
    'You are an expert at reading receipts and invoices. '
    'Extract the information from the receipt image into valid JSON that follows this schema. '
    'Copy text and numbers exactly as they are printed. '
    'Leave a key out if the information is not visible. '
    'Return only the JSON, with no explanation. Schema: '
    + json.dumps(schema_dict)
)
```

Then, run model using this function

```
def run_model(image, max_new_tokens=1024):
    img = preprocess(image)
    messages = [
      {
        'role': 'user',
        'content': [{'type': 'image'}, {'type': 'text', 'text': instruction}]
      }
    ]
    text = processor.apply_chat_template(
      messages, tokenize=False, add_generation_prompt=True
    )
    inputs = processor(text=[text], images=[img], return_tensors='pt').to('cuda')
    with torch.no_grad():
        if USE_ASSEMBLED_GENERATE:
            with torch.autocast('cuda', dtype=torch.float16):
                new = assembled.generate(
                  **inputs, max_new_tokens=max_new_tokens,
                  eos_token_ids=EOS_IDS, repetition_penalty=REPETITION_PENALTY)[0]
        else:
            out = gen_model.generate(
              **inputs, max_new_tokens=max_new_tokens,
              use_cache=True, do_sample=False,repetition_penalty=REPETITION_PENALTY
            )
            new = out[0][inputs['input_ids'].shape[1]:]
    return processor.tokenizer.decode(new, skip_special_tokens=True)
```

Exploration: Fine tune model (1)

Fine tune the model using LORA (Low Rank Adapter) strategy from PEFT.

3,784,556,544
params



29,933,568
params

For training it, use SFTTrainer module from TRL. I modify it a little bit to handle NaN loss value.

```python
class NaNSafeSFTTrainer(SFTTrainer):
    skipped = 0
    consecutive_bad = 0

    def __init__(self, *args, assembled_model, **kwargs):
        super().__init__(*args, **kwargs)
        self.assembled_model = assembled_model
        self.model_accepts_loss_kwargs = False

    def compute_loss(self, model, inputs, return_outputs=False, **kwargs):
        loss = self.assembled_model(**inputs)
        if torch.isfinite(loss):
            self.consecutive_bad = 0
        else:
            self.skipped += 1
            self.consecutive_bad += 1
            if self.consecutive_bad >= 20:
                self.control.should_training_stop = True
            dummy = sum(p.sum() for p in model.parameters() if p.requires_grad) * 0.0
            loss = dummy
        return (loss, {'loss': loss}) if return_outputs else loss
```

Exploration: Fine tune model (2)

Set the configuration for SFT using SFTConfig from TRL.

```
args = SFTConfig(
    output_dir='trl_out', per_device_train_batch_size=1, gradient_accumulation_steps=GRAD_ACCUM,
    num_train_epochs=EPOCHS, max_steps=MAX_STEPS, learning_rate=LEARNING_RATE, lr_scheduler_type='linear',
    warmup_steps=WARMUP_STEPS, max_grad_norm=MAX_GRAD_NORM, optim='adamw_torch', weight_decay=0.01,
    fp16=True, bf16=False, logging_steps=1, save_strategy='no', report_to='none', seed=SEED,
    remove_unused_columns=False, dataset_kwargs={'skip_prepare_dataset': True},
    gradient_checkpointing=False, dataloader_num_workers=0, dataloader_pin_memory=False,
)
```

Then.. train the model.

```
trainer = NaNSafeSFTTrainer(
  model=peft_model, args=args, train_dataset=train_rows, data_collator=collate, processing_class=processor
)

trainer.train()
```

Result: Fine Tune Training



Training time: 96.4 min
Peak VRAM: 4.74 GB
Skipped receipts: 52

field_accuracy	*total_price_match*
80.94	**89.47**
Exact field match accuracy	Total price only accuracy

failure_count	*sec_per_receipt*
2	**23.56**
Total count of failed JSON produce records	Time (seconds) to produce the result

Result: Comparison

